

```
0000 with TEXT_IO; use TEXT_IO;
1 .. 128 ); LIST_FILE_LEN := 0; LIST_FILE := FILE_TYPE; PROCESSED_FILE_STR := STRING(1 .. 256 ); PROCESSED_FILE_STR_LEN := NATU
L := 0; -- Compteurs globaux pour le rapport final TOTAL_FILES := NATURAL; TOTAL_IN_CHARS := NATURAL; TOTAL_OUT_CHARS := NATU
R := 0; function STRIP_LEADING_DOTDOT( PATHED_FILE_NAME : STRING ) return STRING is
rime les sequences initiales "/" et "/" d'un chemin, -- desorte que "/./src/foo/bar.adb" devienne "src/foo/bar.adb". ID := NATURAL;
PATHED_FILE_NAME'FIRST; begin loop if ID + 2 <= PATHED_FILE_NAME'LAST and then PATHED_FILE_NAME( ID .. ID+2 ) = "/" then ID := ID + 2; el
ID := ID + 3; elsif ID + 1 <= PATHED_FILE_NAME'LAST and then PATHED_FILE_NAME( ID .. ID+1 ) = "/" then ID := ID + 1; else
se; end if; end loop; return PATHED_FILE_NAME( ID .. PATHED_FILE_NAME'LAST ); end STRIP_LEADING_DOTDOT;
0010 in out NATURAL; is NATURAL; IN_BUFFER := STRING(1 .. 1024); IN_LEN := NATURAL; OUT_BUFFER := STRING(1 .. 1024); O
UT_LEN := NATURAL; -- Etat de l'analyseur lexical, persistant d'une ligne a l'autre. -- (inutile en Ada car chaines et commentaires ne
franchissent pas) -- Les lignes, mais on garde la variable par prudence. IN_STRING := BOOLEAN; -- Traite une sequence de blan
cs (espaces et tabulations) commençant -- a la colonne visuelle VIS_COL_IN et représentant un saut visuel -- jusqu'a la colonne VIS_COL_OUT. R
egenere dans OUT_BUFFER le blanc -- minimal : tabulations jusqu'au dernier multiple de TAB_WIDTH -- inferieur ou egal a VIS_COL_OUT, puis espa
ces pour completer. procedure EMIT_WHITESPACE( VIS_COL_IN, VIS_COL_OUT : in NATURAL ) is
TURAL := VIS_COL_IN; NEXT_TAB := NATURAL; begin -- Emettre des tabulations tant qu'elles font progresser vers la cible loop
-- Colonne atteinte par une tabulation depuis CUR_COL NEXT_TAB := ((CUR_COL / TAB_WIDTH) + 1) * TAB_WIDTH; exit when NEXT_TAB > VIS
_COL_OUT; OUT_LEN := OUT_LEN + 1; OUT_BUFFER(OUT_LEN) := ' '; CUR_COL := NEXT_TAB; end loop;
avec des espaces while CUR_COL < VIS_COL_OUT loop OUT_LEN := OUT_LEN + 1; OUT_BUFFER(OUT_LEN) := ' '; CUR_COL := CUR
_COL + 1; end loop; end EMIT_WHITESPACE;
0020 TURAL := 1; VIS_COL := NATURAL; -- colonne visuelle courante ORIG_OUT_LEN := NATURAL; begin OUT_LEN := 0; while I
CAR < IN_LEN loop if IN_STRING then -- Dans une chaine litterale : on recopie tout a l'identique OUT_LEN := OUT_LEN + 1; OUT_BUFFER( O
UT_LEN) := IN_BUFFER( ICAR ); if IN_BUFFER( ICAR ) = '"' then -- Un " double dans une chaine represente un guillemet; -- il ne ferme donc pas
la chaine. if ICAR < IN_LEN and then IN_BUFFER( ICAR + 1 ) = '"' then OUT_LEN := OUT_LEN + 1; OUT_BUFFER(OUT_LEN) := '"'; VIS_COL := VIS
_COL + 2; ICAR := ICAR + 2; else IN_STRING := FALSE; VIS_COL := VIS_COL + 1; ICAR := ICAR + 1; end if; else VIS_COL := VIS_COL + 1; I
CAR := ICAR + 1; end if; elsif IN_BUFFER( ICAR ) = ' ' or else IN_BUFFER( ICAR ) = ASCII.HT then -- Debut d'une sequence de blancs hors c
haine litterale : -- on calcule la colonne visuelle atteinte en fin de sequence, puis on regenere le blanc minimal. declare VIS_START : constan
t NATURAL := VIS_COL; VIS_END := NATURAL; while ICAR < IN_LEN and then IN_BUFFER( ICAR ) = ' ' or else IN_BUFFER( ICAR ) =
ASCII.HT loop if IN_BUFFER( ICAR ) = ' ' then VIS_END := VIS_END + 1; else -- tabulation : avance jusqu'au prochain multiple de TA
B_WIDTH VIS_END := ((VIS_END / TAB_WIDTH) + 1) * TAB_WIDTH; end if; ICAR := ICAR + 1; end loop; -- Si la ligne ne contient que des blanc
s (ICAR > IN_LEN), -- on n'emet rien : la ligne sera vide. if ICAR < IN_LEN then EMIT_WHITESPACE( VIS_START, VIS_END ); VIS_COL := VIS_END;
end if; else -- Caractere ordinaire : on le recopie et on avance d'une colonne OUT_LEN := OUT_LEN + 1; OUT_BUFFER(OUT_LEN) := IN
_BUFFER( ICAR ); -- Chaine litterale : un " ouvre une chaine, sauf si le " est -- un caractere litteral " (entoure d'apostrophes). if IN_BUFFER(
ICAR ) = '"' then if not (ICAR >= 2 and then ICAR < IN_LEN and then IN_BUFFER( ICAR - 1 ) = '"') then IN_BUFFER( ICAR + 1 ) := '"'; then
IN_STRING := TRUE; end if; end if; end if; VIS_COL := VIS_COL + 1; ICAR := ICAR + 1; end if; -- Une chaine litterale n
on fermee en fin de ligne est une erreur -- de syntaxe en Ada ; on remet l'etat a FALSE par securite. IN_STRING := FALSE; -- Calcul
de la longueur de la ligne d'origine apres suppression -- des blancs de fin, pour comparer avec la ligne optimisee. ORIG_OUT_LEN := IN_LE
N; while ORIG_OUT_LEN > 0 and then IN_BUFFER( ORIG_OUT_LEN ) = ' ' or else IN_BUFFER( ORIG_OUT_LEN ) = ASCII.HT loop
ORIG_OUT_LEN := ORIG_OUT_LEN - 1; end loop; -- Emettre la ligne optimisee PUT_LINE( FILE_OUT, OUT_BUFFER( 1 .. ORIG_OUT_LEN ) );
-- Statistiques : caracteres lus (ligne d'origine complete) et emis IN_CHARS := IN_CHARS + IN_LEN + 1; -- +1 pour le retour ligne OUT_C
HARS := OUT_CHARS + OUT_LEN + 1; -- Ligne modifiee ? if OUT_LEN < ORIG_OUT_LEN then LINES_CHANGED := LINES_CHANGED + 1; e
lse for K in 1 .. OUT_LEN loop if OUT_BUFFER( K ) /= IN_BUFFER( K ) then LINES_CHANGED := LINES_CHANGED + 1; end if; end
loop; end if; end PROCESS_LINE; end loop; end OPTIMIZE_FILE;
0040 HARS := OUT_CHARS + OUT_LEN + 1; -- Ligne modifiee ? if OUT_LEN < ORIG_OUT_LEN then LINES_CHANGED := LINES_CHANGED + 1; e
lse for K in 1 .. OUT_LEN loop if OUT_BUFFER( K ) /= IN_BUFFER( K ) then LINES_CHANGED := LINES_CHANGED + 1; end if; end
loop; end if; end PROCESS_LINE; end loop; end OPTIMIZE_FILE;
E( LIST_FILE_STR, LIST_FILE_LEN ); OPEN( LIST_FILE, IN_FILE, LIST_FILE_STR( 1 .. LIST_FILE_LEN ) ); NEW_LINE; PUT_LINE( "FICHER
IN OUT GAIN LIGNES" ); PUT_LINE( "-----" );
while not END_OF_FILE( LIST_FILE ) loop GET_LINE( LIST_FILE, PROCESSED_FILE_STR, PROCESSED_FILE_STR_LE
N ); if PROCESSED_FILE_STR_LEN /= 0 and then PROCESSED_FILE_STR( 1 ) = '#' then -- PROCESS_ONE_FILE; declare PR
OCESSED_FILE := FILE_TYPE; OPTIMIZED_FILE := FILE_TYPE; IN_CHARS := NATURAL; OUT_CHARS := NATURAL; LINES_CHA
NGED := NATURAL; OPEN_OK := BOOLEAN; CREATE_OK := BOOLEAN;
0050 N_FILE, PROCESSED_FILE_STR( 1 .. PROCESSED_FILE_STR_LEN ) ); OPEN_OK := TRUE; exception when NAME_ERROR => PUT_LINE( "FICHER SOURCE INEXIS
TANT " & PROCESSED_FILE_STR( 1 .. PROCESSED_FILE_STR_LEN ) ); end; if OPEN_OK then declare REL_PATH := constant STRING; STR
IP_LEADING_DOTDOT( PROCESSED_FILE_STR( 1 .. PROCESSED_FILE_STR_LEN ) ); OUT_FILE_STR := constant STRING := "/OPT_SRC/" & REL_PATH; begin CR
EATE( OPTIMIZED_FILE, OUT_FILE, OUT_FILE_STR ); CREATE_OK := TRUE; exception when NAME_ERROR => PUT_LINE( "FICHER SORTIE NAME_ERROR " & O
UT_FILE_STR ); when USE_ERROR => PUT_LINE( "FICHER SORTIE USE_ERROR " & OUT_FILE_STR ); & " (repertoire inexistant ou droits insuffisants
) ); end; if CREATE_OK then OPTIMIZE_FILE( PROCESSED_FILE, OPTIMIZED_FILE, IN_CHARS, OUT_CHARS, LINES_CHANGED ); CLOSE( OPTIMIZED_FILE
); -- Rapport par fichier declare GAIN : constant INTEGER := INTEGER( IN_CHARS ) - INTEGER( OUT_CHARS ); PCT := INTEGER( 0 ); N
AME_FIELD := STRING( 1 .. 53 ); (others => ' '); NAME_LEN := NATURAL := REL_PATH'LENGTH; begin if IN_CHARS > 0 then PCT := (GAIN
* 100) / INTEGER( IN_CHARS ); end if; if NAME_LEN > NAME_FIELD'LENGTH then NAME_LEN := NAME_FIELD'LENGTH; end if; NAME_F
IELD( 1 .. NAME_LEN ) := REL_PATH( REL_PATH'FIRST .. REL_PATH'FIRST + NAME_LEN - 1 ); PUT( NAME_FIELD ); IN_STR := constant
STRING := NATURAL'IMAGE( IN_CHARS ); OUT_STR := constant STRING := NATURAL'IMAGE( OUT_CHARS ); GAIN_STR := constant STRING := INTEGER'IMA
GE( GAIN ); PCT_STR := constant STRING := INTEGER'IMAGE( PCT ); LINES_STR := constant STRING := NATURAL'IMAGE( LINES_CHANGED ); begin
PUT( ( 1 .. 9 - IN_STR'LENGTH => ' ') & IN_STR ); PUT( ( 1 .. 9 - OUT_STR'LENGTH => ' ') & OUT_STR ); PUT( ( 1 .. 9 - GAIN_STR'LENG
TH => ' ') & GAIN_STR ); PUT( ( 1 .. 5 - PCT_STR'LENGTH => ' ') & PCT_STR ); PUT( ( 1 .. 8 - LINES_STR'LENGTH => ' ') & LINES_STR );
NEW_LINE; end; end; TOTAL_FILES := TOTAL_FILES + 1; TOTAL_IN_CHARS := TOTAL_IN_CHARS + IN_CHARS; TOTAL_OUT_CHARS := TOTAL_OUT_CHA
R + OUT_CHARS; end if; end; CLOSE( PROCESSED_FILE ); end if; end PROCESS_ONE_FILE; end if; end loop; end CLO
SE( LIST_FILE ); -- Rapport global NEW_LINE; PUT_LINE( "-----" ); declare TOTAL_GAIN : constant INTEGER := INTEGER( TOTAL_IN_CHARS ) - INTEGER( TOTAL_OUT_CHARS ); TOTAL_PCT := INTEGER( 0 ); begin
if TOTAL_IN_CHARS > 0 then TOTAL_PCT := (TOTAL_GAIN * 100) / INTEGER( TOTAL_IN_CHARS ); end if; PUT_LINE( "TOTAL
" & NATURAL'IMAGE( TOTAL_FILES ) & " fichier(s) traite(s)" ); PUT_LINE( "CARACTERES IN : " & NATURAL'IMAGE( TOTAL_IN_CHARS ) ); PUT_LINE( "CARACTERE
S OUT : " & NATURAL'IMAGE( TOTAL_OUT_CHARS ) ); PUT_LINE( "GAIN : " & INTEGER'IMAGE( TOTAL_GAIN ) & " octets (" & INTEGER'IMAGE( TOTAL_PCT ) & "%)" ); end; exception when NAME_ERROR => PUT_LINE( "FICHER LISTE INEXISTANT " & LIST_FILE_STR( 1 .. LIST_FILE_LEN ) ); end OPTIM
_TEXT;
```